



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	TE-AI&DS	Semester:	V
Course Code:	2015111	Course Name:	Statistics for Machine Learning and Data Science Lab

Name of Student:	Yash Vilas Karande
Roll No. :	17
Experiment No.:	9
Title of the Experiment:	Evaluate and Interpret Machine Learning Models using Validation and Explainability Technique
Date of Performance:	23/09/26
Date of Submission:	30/09/26

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : **Dr. Tatwadarshi P.N.**

Signature :

Date :



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Aim: To apply model evaluation, Bayesian inference concepts, and explainable AI techniques to interpret and understand machine learning predictions on the Pima Indians Diabetes Dataset.

Objectives

After completing this experiment, the student will be able to:

1. Apply model evaluation and Bayesian concepts to analyze the reliability of machine learning predictions.
2. Apply explainable AI techniques to interpret model predictions and identify important features influencing the results.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Scikit-learn
- SHAP
- LIME

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains diagnostic information for **768 female patients**.

The target variable is: Outcome = 0: Non-diabetic and Outcome = 1: Diabetic

Procedure



1. Load and preprocess the Pima Indians Diabetes Dataset.
2. Divide the dataset into training and testing sets.
3. Train a suitable classification model.
4. Evaluate the model using k-fold cross-validation.
5. Calculate suitable performance measures such as accuracy and F1-score.
6. Calculate the Brier Score and generate a calibration curve.
7. Explain selected model predictions using SHAP or LIME.
8. Identify the important features influencing the predictions.
9. Interpret and document the model evaluation and explainability results.

Description of the Experiment

A machine learning model should not be evaluated only on its performance on a single test split. Cross-validation provides a more reliable estimate of model performance.

The experiment follows: Dataset, Train Model, Cross-Validation, Performance Evaluation, Calibration Analysis, Explainable AI and Interpretation

The experiment combines three aspects:

- **Model Evaluation:** How well does the model perform?
- **Calibration:** How reliable are the predicted probabilities?
- **Explainability:** Why did the model make a particular prediction?

Detailed Description of Statistical Methods

k-Fold Cross-Validation

The dataset is divided into k subsets.

The model is trained on $k - 1$ subsets and tested on the remaining subset.

This process is repeated k times.

The final performance is calculated using the average performance across all folds.



Bias-Variance Trade-Off

A model should balance:

- **Bias:** Error due to an overly simple model.
- **Variance:** Error due to a model being too sensitive to the training data.

A good model attempts to achieve a suitable balance between bias and variance.

Calibration Curve

A calibration curve compares:

Predicted Probability

with:

Observed Frequency

A well-calibrated model produces probabilities that closely match the actual frequency of the outcome.

Brier Score

The Brier Score measures the accuracy of probabilistic predictions:

$$BS = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2$$

where:

- p_i = predicted probability
- y_i = actual outcome

A lower Brier Score generally indicates better probabilistic predictions.

SHAP

SHAP values explain the contribution of individual features to a model prediction.

They help answer:



Which features contributed to increasing or decreasing the predicted probability?

LIME

LIME explains an individual prediction by creating small variations around the observation and approximating the model locally using an interpretable model.

Implementation

```
[1] ✓ 14s !pip install shap lime -q
```

275.7/275.7 kB 5.7 MB/s eta 0:00:00

Preparing metadata (setup.py) ... done
Building wheel for lime (setup.py) ... done

```
[2] ✓ 9s import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    brier_score_loss,
    classification_report
)
from sklearn.calibration import calibration_curve

import shap
from lime.lime_tabular import LimeTabularExplainer
```

```
[3] ✓ 0s url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

columns = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age",
    "Outcome"
]

df = pd.read_csv(url, names=columns)

print("Dataset Shape:", df.shape)
df.head()
```

Dataset Shape: (768, 9)

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
[4]
✓ Os
print(df.info())
print("\nMissing values:")
print(df.isnull().sum())

print("\nOutcome distribution:")
print(df["Outcome"].value_counts())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Pregnancies           768 non-null    int64
 1   Glucose               768 non-null    int64
 2   BloodPressure         768 non-null    int64
 3   SkinThickness         768 non-null    int64
 4   Insulin               768 non-null    int64
 5   BMI                   768 non-null    float64
 6   DiabetesPedigreeFunction 768 non-null    float64
 7   Age                   768 non-null    int64
 8   Outcome               768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None

Missing values:
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
DiabetesPedigreeFunction 0
Age               0
Outcome           0
dtype: int64

Outcome distribution:
Outcome
0      500
1      268
Name: count, dtype: int64

[5]
✓ Os
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

print("Features:")
print(X.columns.tolist())

print("\nTarget:")
print(y.name)

Features:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']

Target:
Outcome

[6]
✓ Os
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

Training samples: 614
Testing samples: 154

[7]
✓ Os
model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(random_state=42, max_iter=1000))
])

model.fit(X_train, y_train)

print("Model training completed successfully.")

Model training completed successfully.
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

[8] ✓ Os	<pre>y_pred = model.predict(X_test) y_prob = model.predict_proba(X_test)[: , 1] print("First 10 predicted classes:") print(y_pred[:10]) print("\nFirst 10 predicted probabilities:") print(y_prob[:10])</pre>
✓	<pre>... First 10 predicted classes: [1 0 0 0 0 0 0 1 0 1] First 10 predicted probabilities: [0.61667353 0.11278116 0.27253958 0.28668164 0.00448143 0.18904882 0.46842913 0.91697653 0.10324589 0.78113127]</pre>
[9] ✓ Os	<pre>accuracy = accuracy_score(y_test, y_pred) f1 = f1_score(y_test, y_pred) print("Accuracy:", accuracy) print("F1-score:", f1) print("\nClassification Report:") print(classification_report(y_test, y_pred))</pre>
✓	<pre>... Accuracy: 0.7142857142857143 F1-score: 0.56 Classification Report: precision recall f1-score support 0 0.76 0.82 0.79 100 1 0.61 0.52 0.56 54 accuracy 0.71 macro avg 0.68 weighted avg 0.71</pre>
[10] ✓ Os	<pre>cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) cv_accuracy = cross_val_score(model, X, y, cv=cv, scoring="accuracy") print("Accuracy scores for each fold:") print(cv_accuracy) print("\nMean Accuracy:", cv_accuracy.mean()) print("Standard Deviation:", cv_accuracy.std())</pre>
✓	<pre>... Accuracy scores for each fold: [0.77922078 0.79220779 0.78571429 0.76470588 0.75163399] Mean Accuracy: 0.7746965452847806 Standard Deviation: 0.014698285678615699</pre>
[11] ✓ Os	<pre>cv_f1 = cross_val_score(model, X, y, cv=cv, scoring="f1") print("F1-score for each fold:") print(cv_f1) print("\nMean F1-score:", cv_f1.mean()) print("Standard Deviation:", cv_f1.std())</pre>
✓	<pre>... F1-score for each fold: [0.63829787 0.65217391 0.64516129 0.61702128 0.64150943] Mean F1-score: 0.6388327572528987 Standard Deviation: 0.011842534839940536</pre>
[12] ✓ Os	<pre>brier = brier_score_loss(y_test, y_prob) print("Brier Score:", brier)</pre>
✓	<pre>Brier Score: 0.16793272823586033</pre>



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

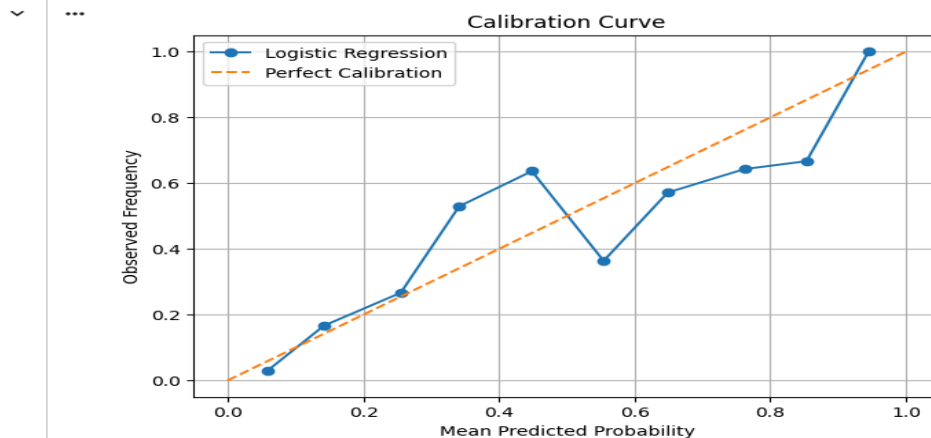
```
[13] ✓ Os
prob_true, prob_pred = calibration_curve(
    y_test,
    y_prob,
    n_bins=10
)

plt.figure(figsize=(7, 5))

plt.plot(
    prob_pred,
    prob_true,
    marker="o",
    label="Logistic Regression"
)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Perfect Calibration"
)

plt.xlabel("Mean Predicted Probability")
plt.ylabel("Observed Frequency")
plt.title("Calibration Curve")
plt.legend()
plt.grid()
plt.show()
```



```
[14] ✓ Os
scaler = model.named_steps["scaler"]
classifier = model.named_steps["classifier"]

X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

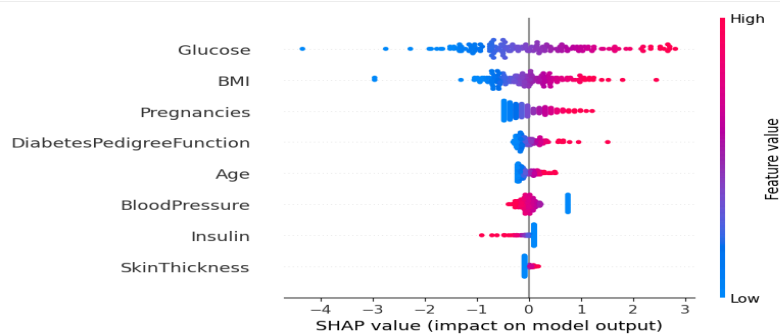
explainer = shap.LinearExplainer(
    classifier,
    X_train_scaled
)

shap_values = explainer(X_test_scaled)

print("SHAP explanation generated successfully.")
```

WARNING:shap:Background dataset has 614 samples but max_samples=100. Subsampling to 100 samples for SHAP value computation. To use all samples, set max_samples=614 when initializing the masker.
SHAP explanation generated successfully.

```
[15] ✓ Os
shap.summary_plot(
    shap_values,
    X_test_scaled,
    feature_names=X.columns
)
```

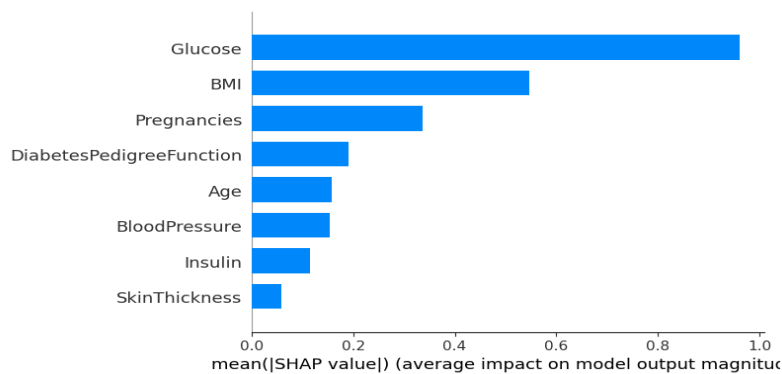




Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
[16]
✓ Os
shap.summary_plot(
    shap_values,
    X_test_scaled,
    feature_names=X.columns,
    plot_type="bar"
)
```

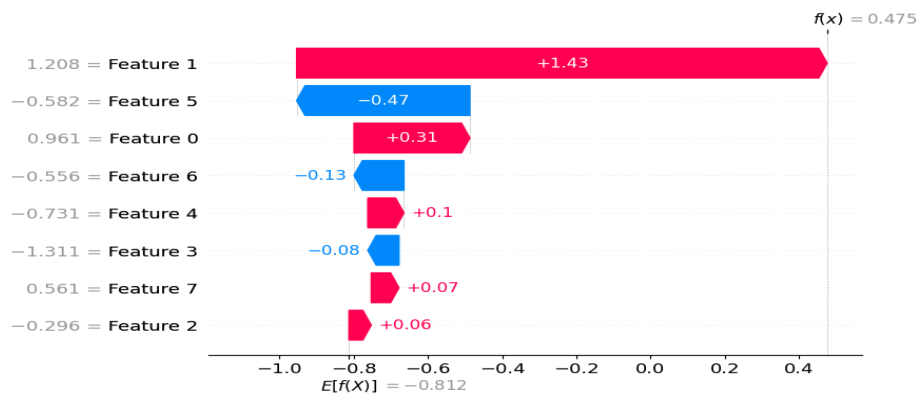


```
[17]
✓ Os
sample_index = 0

print("Actual Outcome:", y_test.iloc[sample_index])
print("Predicted Outcome:", y_pred[sample_index])
print("Predicted Probability:", y_prob[sample_index])
```

Actual Outcome: 0
Predicted Outcome: 1
Predicted Probability: 0.6166735258049927

```
[18]
✓ Ts
shap.plots.waterfall(shap_values[sample_index])
```



```
[19]
✓ Os
mean_shap = np.abs(shap_values.values).mean(axis=0)

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Mean |SHAP value|": mean_shap
})

feature_importance = feature_importance.sort_values(
    "Mean |SHAP value|",
    ascending=False
)

print(feature_importance)
```

	Feature	Mean SHAP value
1	Glucose	0.961686
5	BMI	0.547771
0	Pregnancies	0.337378
6	DiabetesPedigreeFunction	0.190496
7	Age	0.157429
2	BloodPressure	0.154123
4	Insulin	0.115287
3	SkinThickness	0.059255



Conclusion

The classification model was evaluated using cross-validation and calibration analysis to understand its performance and reliability. Explainable AI techniques such as SHAP or LIME were applied to identify the features influencing individual predictions. The experiment demonstrated that model performance, probability reliability, and interpretability are important aspects of evaluating machine learning systems.

Questions to be Answered by Students

1. What was the mean and standard deviation of the model performance across the cross-validation folds?

ANS: The model was evaluated using 5-fold stratified cross-validation. The mean accuracy obtained across the five folds was **0.7747**, with a standard deviation of **0.0147**. The relatively low standard deviation indicates that the model's accuracy remained fairly consistent across the different cross-validation folds.

2. What was the obtained Brier Score, and what does it indicate about the reliability of the predicted probabilities?

ANS: The obtained Brier Score was **0.1679**. The Brier Score measures the difference between the predicted probabilities and the actual outcomes. A lower Brier Score indicates that the predicted probabilities are closer to the actual outcomes. Thus, the obtained score indicates that the model produced reasonably reliable probabilistic predictions, although the probabilities were not perfectly accurate.

3. Was the model well calibrated based on the calibration curve? Justify your answer.

ANS: The model was **not perfectly calibrated** based on the calibration curve. The calibration curve shows noticeable deviations from the diagonal line representing perfect calibration, particularly in the middle probability ranges. This indicates that the predicted probabilities do not always closely match the observed frequencies. Therefore, the model provides useful probability estimates but shows some calibration error.

4. Which features were identified as the most influential using SHAP or LIME?

ANS: According to the SHAP feature-importance analysis, the most influential features were **Glucose, BMI, and Pregnancies**. Their mean absolute SHAP values were **0.961686, 0.547771, and 0.337378**, respectively. This indicates that these features had a greater influence on the model's predictions compared with the other features in the dataset. Among them, **Glucose** had the highest influence.

5. Explain one individual prediction using the obtained explainability results.

ANS: For the selected test observation, the actual outcome was **0 (Non-diabetic)**, while the model predicted **1 (Diabetic)** with a predicted probability of **61.67%**. The SHAP analysis showed that **Glucose** had the strongest positive contribution with a SHAP value of **+1.428766**, followed by **Pregnancies** with **+0.312161**. These features increased the prediction towards the diabetic class. On the other hand, **BMI** had a negative contribution of



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

–**0.466601**, which pushed the prediction towards the non-diabetic class. Overall, the positive contributions, particularly from Glucose and Pregnancies, led the model to predict the patient as diabetic despite the actual outcome being 0.